

Module 5 Activity

June 19, 2026

Module 5 Activity

Assigned at the start of Module 5

Due at the end of Module 5

Unnas Hussain

1 Weekly Discussion Forum Participation

Each week, you are required to participate in the module's discussion forum. The discussion forum consists of the week's Module Activity, which is released at the beginning of the module. You must complete/attempt the activity before you can post about the activity and anything that relates to the topic.

1.1 Grading of the Discussion

1.1.1 1. Initial Post:

Create your thread by **Day 5 (Saturday night at midnight, PST)**.

1.1.2 2. Responses:

Respond to at least two other posts by **Day 7 (Monday night at midnight, PST)**.

1.2 Grading Criteria:

Your participation will be graded as follows:

1.2.1 Full Credit (100 points):

- Submit your initial post by **Day 5**.
- Respond to at least two other posts by **Day 7**.

1.2.2 Half Credit (50 points):

- If your initial post is late but you respond to two other posts.
- If your initial post is on time but you fail to respond to at least two other posts.

1.2.3 No Credit (0 points):

- If both your initial post and responses are late.
 - If you fail to submit an initial post and do not respond to any others.
-

1.3 Additional Notes:

- **Late Initial Posts:** Late posts will automatically receive half credit if two responses are completed on time.
 - **Substance Matters:** Responses must be thoughtful and constructive. Comments like “Great post!” or “I agree!” without further explanation will not earn credit.
 - **Balance Participation:** Aim to engage with threads that have fewer or no responses to ensure a balanced discussion.
-

1.4 Avoid:

- A number of posts within a very short time-frame, especially immediately prior to the posting deadline.
- Posts that complement another post, and then consist of a summary of that.

2 Activity: Extending and Analyzing the Expectation-Maximization Algorithm

This activity is designed to deepen your understanding of the Expectation-Maximization (EM) algorithm and its computational efficiency. Below, you will find a code implementation of the EM algorithm for a Gaussian Mixture Model. Your task is twofold:

2.1 Task 1: Extend the Algorithm

Modify the code to run over multiple iterations. Implement: - **A maximum iteration count** to limit the number of iterations. - **A stopping criterion** based on convergence (e.g., a small change in parameter values between iterations).

2.2 Task 2: Analyze Runtime

Add line-by-line runtime comments to the code to measure the execution time of each section. Using these measurements, derive the **runtime complexity** of the algorithm in terms of the number of data points $((n))$ and components $((k))$.

2.3 Instructions

2.3.1 Part 1: Extending the Algorithm

1. Add a **for** loop or **while** loop to iterate over the Expectation-Maximization steps until:
 - A **maximum number of iterations** is reached (e.g., 100).

- The parameter values (`mu`, `sigma`, `pk`) converge, i.e., the change in values is below a predefined threshold (e.g., (10^{-4})).
2. Modify the algorithm to check for convergence at the end of each iteration:
 - Use a metric like the **norm of the difference in means** (`mu`) or the **log-likelihood** of the data.
 - Print the number of iterations the algorithm required before convergence.
-

2.3.2 Part 2: Analyzing Runtime

1. Import Python's `time` module and measure the runtime of each section of the code:
 - **Initialization**: Measure the time to compute the initial `mu`, `sigma`, and `pk`.
 - **E-Step**: Measure the time to compute probabilities and numerators.
 - **M-Step**: Measure the time to update the parameters.
2. Add comments next to each runtime measurement to document how long the operation took.
3. Using the runtimes, analyze the algorithm's complexity:
 - Consider (`n`), the number of data points.
 - Consider (`k`), the number of Gaussian components.
 - Derive the overall runtime complexity as a function of (`n`) and (`k`).

```
[4]: import numpy as np
import math
import time

MAX_ITERS = 100
CONVERGANCE_DELTA_THRESH = 1e-3

# Define the initial data columns
column_1 = np.array([1, 4, 1, 4])
column_2 = np.array([2, 2, 3, 3])

# Create a 2-column dataset
x = np.column_stack((column_1, column_2))
print(x)

# Initialization timing start
init_start = time.time()

# Compute the means and sample standard deviations of each column
column_means = np.mean(x, axis=0)
print(column_means)

column_stddevs = np.std(x, axis=0, ddof=1) # Sample standard deviation (ddof=1)
```

```

print(column_stddevs)

# Compute the average of the standard deviations across columns
average_of_values = np.mean(column_stddevs)

# Create an array of uniform standard deviations using the average value
std_deviations = np.full_like(column_stddevs, average_of_values)
print(std_deviations)

# Initialize means (mu) based on the data
mu = ((column_means.reshape(1, -1) * np.array([1, 1]).reshape(-1, 1)) +
      (column_stddevs.reshape(1, -1) * np.array([-0.1867, 0.7257]).reshape(-1, 1)))
print(mu)

# Initialize standard deviations (sigma)
sigma = std_deviations.reshape(1, -1) * np.array([1, 1]).reshape(1, -1)
print(sigma)

# Initialize the prior probabilities (pk)
pk = np.array([1, 1]).reshape(1, -1) / 2
print(pk)

# Initialization timing end
# Last Measured: 0.0007061958312988281
init_time = time.time() - init_start

# Define the Gaussian probability density function
def g(x, mu, sigma):
    temp = 1 / (((2 * math.pi) ** 0.5) * sigma) ** 2) # Gaussian
    ↪normalization constant
    temp2 = (np.linalg.norm(x - mu) / sigma) ** 2 # Squared Mahalanobis
    ↪distance
    temp3 = np.exp(-0.5 * temp2) # Exponential factor
    return temp * temp3

# Helper function: Sum every k-th value in a list
def sum_every_kth_value_list(arr, k):
    result = []
    for i in range(k):
        sum_val = sum(arr[i::k])
        result.append(sum_val)
    return result

# Helper function: Sum values in chunks of size k
def sum_every_k_values(arr, k):
    if k <= 0 or len(arr) % k != 0:

```

```

        return "Invalid input"
    return [sum(arr[i:i + k]) for i in range(0, len(arr), k)]

# Helper function: Get the n-th set of k values from a list
def get_nth_set_of_k_values(arr, k, n):
    if k <= 0 or n <= 0:
        return "Invalid input"
    start_index = (n - 1) * k
    end_index = start_index + k
    return arr[start_index:end_index]

def param_vector(mu, sigma, pk):
    return np.concatenate([np.array(mu).ravel(), np.array(sigma).ravel(), np.
↪array(pk).ravel()])

# Helper function: check if the values are converging by checking norm of ↵
↪difference in params
def check_norm_of_diff(prev_vals, new_vals):
    return np.linalg.norm(new_vals - prev_vals) < CONVERGANCE_DELTA_THRESH

# E & M Steps
converged = False
iterations_left = MAX_ITERS
new_mu = []
new_std = []
updated_p = []
total_e_time = 0
total_m_time = 0

while not converged and iterations_left > 0:
    # E-step timing start
    e_start = time.time()

    # E-step: Calculate numerators for updating probabilities
    numerators = []
    for j in range(0, mu.shape[0]): # Iterate over each Gaussian component
        for i in range(0, x.shape[0]): # Iterate over each data point
            value = g(x[i], mu[j], sigma[0][j]) * pk[0][j]
            numerators.append(value)

    # Compute denominators for normalizing probabilities
    denominators = sum_every_kth_value_list(numerators, int(len(numerators) / ↵
↪2))

    # Update probabilities for each data point and each component
    prev_p = updated_p
    new_p = []

```

```

for i in range(0, len(numerators)):
    new_p.append(numerators[i] / denominators[i % int(len(numerators) / 2)])

# Compute p_k_n (updated prior probabilities)
p_k_n = sum_every_k_values(new_p, int(len(numerators) / 2))

# E-step timing end
# Last Measured: 0.000370025634765625
total_e_time += time.time() - e_start

# M-step timing start
m_start = time.time()

# M-step: Update the means (mu) and standard deviations (sigma)
prev_mu = new_mu
prev_std = new_std
new_mu = []
new_std = []
for i in range(1, 3): # Iterate over components
    temp = 0
    temp2 = 0
    for j in range(0, x.shape[0]): # Update mean
        temp += get_nth_set_of_k_values(new_p, 4, i)[j] * x[j]
    temp = temp / p_k_n[i - 1]
    new_mu.append(temp)

    for k in range(0, x.shape[0]): # Update standard deviation
        temp2 += get_nth_set_of_k_values(new_p, 4, i)[k] * (np.linalg.
↪norm(x[k] - new_mu[i - 1]) ** 2)
        temp2 = (temp2 / (2 * p_k_n[i - 1])) ** 0.5
    new_std.append(temp2)

# Update prior probabilities
updated_p = []
for i in range(0, 2):
    updated_p.append(p_k_n[i] / int(len(numerators) / 2))

# M-step timing end
# Last Measured: 0.0002110004425048828
total_m_time += time.time() - m_start

# Check convergence (skip first iteration since prev values are
↪uninitialized)
if prev_mu:
    prev_param = param_vector(prev_mu, prev_std, prev_p)
    new_params = param_vector(new_mu, new_std, updated_p)
    converged = check_norm_of_diff(prev_param, new_params)

```

```

iterations_left -= 1

print(iterations_left)

# Feed updated parameters back into next iteration
mu = np.array(new_mu)
sigma = np.array(new_std).reshape(1, -1)
pk = np.array(updated_p).reshape(1, -1)

num_iters = MAX_ITERS - iterations_left

# Final updated parameters: new means, new standard deviations, and updated
↳ priors
print('\n=== Part1 ===')
print('Number of Iterations:', num_iters)
print(':', new_mu)
print(':', new_std)
print('p:', updated_p)

# Runtime analysis
print('\n=== Part2 ===')
print('Initialization time:', init_time) #  $O(n)$ 
print('Total E-step time:', total_e_time) #  $O(n * k)$  per iteration
print('Total M-step time:', total_m_time) #  $O(n * k)$  per iteration
print('Overall complexity:  $O(T * n * k)$  where  $T =$ ', num_iters)

```

```

[[1 2]
 [4 2]
 [1 3]
 [4 3]]
[2.5 2.5]
[1.73205081 0.57735027]
[1.15470054 1.15470054]
[[2.17662611 2.3922087 ]
 [3.75694927 2.91898309]]
[[1.15470054 1.15470054]]
[[0.5 0.5]]

```

```

99
98
97
96

```

```

=== Part1 ===
Number of Iterations: 4
: [array([1. , 2.5]), array([4. , 2.5])]
: [np.float64(0.35355339059327523), np.float64(0.35355339059327523)]
p: [np.float64(0.5), np.float64(0.5)]

```

=== Part2 ===

Initialization time: 0.0007419586181640625

Total E-step time: 0.0003108978271484375

Total M-step time: 0.0002491474151611328

Overall complexity: $O(T * n * k)$ where $T = 4$

The EM algorithm has $O(n \times k)$ complexity per iteration. Initialization runs in $O(n)$ time since it computes means and standard deviations over all data points. The E-step is $O(n \times k)$ because it computes the Gaussian probability for every combination of n data points and k components. The M-step is also $O(n \times k)$ since it updates each of the k means and standard deviations by iterating over all n points. Therefore, the overall runtime complexity is $O(n \times k)$ per iteration, making the algorithm linear in both the number of data points and the number of mixture components.